

Use Case : Projet 3

Rapport

Réalisé par

GARREC Aimie
THERET Sarah

ING 3 DS

Dirigée par Mr Kossi FOLLY

Année 2025-2026

Sommaire

Introduction.....	2
I. Les fonctionnalités.....	2
A. Les questions numériques.....	2
B. L'analyse d'annonce.....	2
C. Le résumé de texte.....	3
D. Le résumé de PDF.....	4
E. La recherche d'informations textuelles.....	5
II. La mise en production.....	5
A. L'API.....	5
B. L'application Streamlit.....	5
C. Le docker deployment ready.....	5
Conclusion.....	6
Bibliographie.....	7

Introduction

Aujourd'hui, le secteur immobilier évolue à grande vitesse. Entre la multiplication des données disponibles, les fluctuations constantes du marché et les attentes toujours plus élevées des clients, il devient essentiel de pouvoir accéder rapidement à des informations fiables et pertinentes. Pourtant, exploiter efficacement ces données reste un vrai défi, surtout lorsqu'elles sont volumineuses, hétérogènes ou peu structurées.

C'est donc dans ce contexte que ce projet s'inscrit. Le but est de créer un assistant intelligent capable de comprendre des questions formulées naturellement et d'y répondre de manière claire et rapide. L'objectif est de proposer un outil adapté aux professionnels de l'immobilier, leur permettant d'accéder rapidement à des informations pertinentes sans avoir à manipuler directement des bases de données complexes. L'idée est de simplifier l'exploitation des données : poser une question de manière naturelle et obtenir une réponse claire, exploitable et directement utile dans un contexte métier.

Au fil de ce rapport, nous allons explorer les différentes étapes de conception de cet assistant : de la préparation des données à la mise en place d'un système de question-réponse, en passant par l'indexation vectorielle, la génération de résumés ou encore l'analyse de sentiment. L'objectif est de construire une solution complète, capable non seulement de répondre à des questions, mais aussi d'apporter une réelle valeur d'aide à la décision.

I. Les fonctionnalités

A. Les questions numériques

On commence par automatiser le téléchargement des fichiers DVF (Données de Valeurs Foncières) pour chaque année étudiée. Cette étape permet de garantir la reproductibilité du traitement tout en évitant les manipulations manuelles, sources d'erreurs et de perte de temps surtout. Tous les fichiers construits de cette façon "dvf_{annee}" sont ainsi récupérés directement depuis les sources officielles [1] et stockés localement afin de pouvoir être exploités dans les étapes suivantes.

```
=====
Résumé: 6 fichiers téléchargés
Années: ['2025', '2024', '2023', '2022', '2021', '2020']
=====
```

Une fois les données téléchargées, chaque fichier DVF, souvent volumineux, est chargé de manière progressive en utilisant une lecture par *chunks* via `pd.read_csv(..., chunksize=...)`. Cette méthode permet d'éviter de charger l'intégralité du fichier en mémoire en une seule fois. Les données sont ainsi lues par portions successives, chacune étant traitée individuellement, puis concaténée afin de reconstituer le dataframe correspondant à l'année. Ce mode de traitement permet d'appliquer les premières opérations (filtrage, transformation) au fur et à mesure de la lecture, ce qui améliore la

stabilité du programme. Une fois l'ensemble des chunks assemblés, les données peuvent ensuite être manipulées plus facilement sous forme de structure tabulaire.

Les données brutes nécessitent ensuite une phase de nettoyage approfondie. Celle-ci comprend notamment la suppression des colonnes non pertinentes, le traitement des valeurs manquantes ainsi que la normalisation des formats, en particulier pour les dates et les variables numériques.

Puis, on utilise Isolation Forest qui permet d'identifier automatiquement les observations aberrantes en se basant sur leur isolement dans l'espace des données : les points atypiques sont plus facilement séparés que les observations normales. L'utilisation de cette approche permet de repérer des transactions immobilières anormalement élevées ou incohérentes, qui pourraient biaiser les analyses. Les valeurs détectées comme anomalies peuvent alors être supprimées ou traitées spécifiquement, contribuant ainsi à améliorer la qualité globale du jeu de données.

```
def nettoyage(df):
    df = df[df['Nature mutation'] == 'Vente']
    df = df[['Date mutation', 'Valeur fonciere', 'Code postal', 'Commune', 'Code departement', 'Code type local', 'Surface reelle bati', 'Surface terrain']]
    df = df.dropna(subset=['Valeur fonciere', 'Code postal', 'Code type local', 'Surface reelle bati']).reset_index(drop=True)
    df = df.drop_duplicates()
    df = df[df['Code type local'].isin([1, 2])]

    df['Date mutation'] = pd.to_datetime(df['Date mutation'], format='%d/%m/%Y', errors='coerce')
    df['Année'] = df['Date mutation'].dt.year
    df['Valeur fonciere'] = df['Valeur fonciere'].astype(str).str.replace(',', '.').str.replace(r'\s+', '', regex=True).astype(float).round().astype(int)
    df['Surface reelle bati'] = df['Surface reelle bati'].astype(int)
    df['Surface terrain'] = df['Surface terrain'].fillna(0).astype(int)
    df['Code type local'] = df['Code type local'].astype(int)
    df.drop(columns=['Date mutation'], inplace=True)

    # Filtres prix et surface minimum
    df = df[(df['Valeur fonciere'] >= 30000) & (df['Surface reelle bati'] >= 15)]

    numeric_df = df[['Valeur fonciere', 'Surface reelle bati', 'Surface terrain']]

    iso_forest = IsolationForest(contamination=0.15, random_state=42)
    predictions = iso_forest.fit_predict(numeric_df)

    df['anomaly'] = predictions
    df_clean = df[predictions == 1]
    df_clean = df_clean.drop(columns=['anomaly'])
    df_clean = df_clean.reset_index(drop=True)
```

Une fois les données nettoyées pour chaque année, les différents dataframes sont concaténés afin de constituer une base de données unique regroupant l'ensemble des transactions sur la période étudiée.

Dans un second temps, la base de données est enrichie par l'ajout d'informations communales issues d'une source ouverte mise à disposition sur la plateforme data.gouv.fr [2]. Le jeu de données utilisé regroupe des informations détaillées sur les communes françaises, notamment la population, les codes administratifs et certaines données de localisation.

L'intégration de ces données repose sur une opération de jointure entre la base DVF et le référentiel communal. Afin d'assurer une correspondance fiable entre les deux sources, un travail de normalisation est effectué en amont sur les noms de communes. Ceux-ci sont

convertis en majuscules et harmonisés pour limiter les écarts liés aux accents, aux tirets ou aux variations d'écriture. La jointure est ensuite réalisée à partir de deux clés : le nom de la commune normalisée et le numéro de département. Cette double condition permet de lever les ambiguïtés liées aux homonymes entre communes situées dans des départements différents.

Cet enrichissement permet d'associer à chaque transaction immobilière des informations contextuelles essentielles, telles que la population ou des indicateurs géographiques. Il devient ainsi possible d'analyser les dynamiques immobilières en tenant compte des caractéristiques territoriales, par exemple en comparant les prix selon la taille des communes ou leur localisation. Cette étape constitue donc un apport fondamental pour donner du sens aux données et approfondir les analyses.

	Valeur foncière	Code postal	Commune	Code département	Code type local	Surface réelle	Surface bati	Surface terrain	Année	Nb_hab_com	Dens_hab_com	latitude_mairie	longitude_mairie
0	72500	10000.0	TROYES	10	2	29		0	2024	62597	4742.20	48.297	4.078
1	250000	10120.0	SAINT ANDRE LES VERGERS	10	1	101		966	2024	12701	2167.41	48.278	4.048
2	250000	10120.0	SAINT ANDRE LES VERGERS	10	1	101		753	2024	12701	2167.41	48.278	4.048
3	138500	10100.0	GELANNES	10	1	105		398	2024	734	60.51	48.475	3.668
4	249100	10000.0	TROYES	10	2	115		0	2024	62597	4742.20	48.297	4.078

Enfin, les données sont restructurées afin de produire des fichiers au format JSON, organisés par commune. Chaque fichier regroupe l'ensemble des informations relatives à une ville donnée, incluant à la fois ses caractéristiques et les transactions immobilières associées. Ce format de sortie est particulièrement adapté à une utilisation dans des applications web ou des outils de visualisation, facilitant ainsi l'exploitation et la diffusion des résultats.

Voici l'exemple du JSON pour la ville de Brest :

```
"BREST|29": {
  "Nb_total_ventes": {"2020": 666, "2021": 3605, "2022": 3393, "2023": 2662, "2024": 2268},
  "Nb_ventes_maisons": {"2020": 156, "2021": 723, "2022": 706, "2023": 589, "2024": 483},
  "Nb_ventes_appartements": {"2020": 510, "2021": 2882, "2022": 2687, "2023": 2073, "2024": 1785},

  "Prix_moy": {"2020": 129908.2, "2021": 148250.48, "2022": 158349.51, "2023": 158286.84},
  "Prix_med": {"2020": 115000.0, "2021": 125000.0, "2022": 136000.0, "2023": 137360.0, "2024": 135000.0},
  "Prix_min": {"2020": 1.0, "2021": 1000.0, "2022": 1.0, "2023": 1000.0, "2024": 120.0},
  "Prix_max": {"2020": 420000.0, "2021": 644000.0, "2022": 873300.0, "2023": 615850.0, "2024": 680000.0},
  "Ecart_type_prix": {"2020": 70743.66, "2021": 85388.92, "2022": 89524.74, "2023": 82609.78},

  "Prix_m2_moy": {"2020": 2014.75, "2021": 2296.15, "2022": 2506.75, "2023": 2503.68, "2024": 2477.2},
  "Prix_m2_med": {"2020": 1751.51, "2021": 1951.22, "2022": 2207.55, "2023": 2192.89, "2024": 2201.21},
  "Prix_m2_min": {"2020": 0.02, "2021": 17.46, "2022": 0.01, "2023": 13.7, "2024": 3.5},
  "Prix_m2_max": {"2020": 10312.5, "2021": 16868.97, "2022": 16111.11, "2023": 18088.24, "2024": 15750.0},

  "Surface_moy_interieur": {"2020": 68.7, "2021": 68.49, "2022": 66.41, "2023": 67.25, "2024": 67.16},
  "Surface_med_interieur": {"2020": 67.0, "2021": 67.0, "2022": 65.0, "2023": 66.0, "2024": 66.0},
  "Surface_moy_terrain": {"2020": 86.95, "2021": 78.23, "2022": 73.09, "2023": 80.14, "2024": 78.81},
  "Surface_med_terrain": {"2020": 0.0, "2021": 0.0, "2022": 0.0, "2023": 0.0, "2024": 0.0},

  "Prix_moy_maison": {"2020": 179083.16, "2021": 216068.35, "2022": 229592.58, "2023": 231814.12, "2024": 231814.12},
```

```

"Prix_med_maison": {"2020": 168943.5, "2021": 204000.0, "2022": 220000.0, "2023": 220900.0},
"Prix_m2_moy_maison": {"2020": 1986.68, "2021": 2336.5, "2022": 2542.23, "2023": 2563.98},
"Prix_m2_med_maison": {"2020": 1848.72, "2021": 2246.38, "2022": 2447.4, "2023": 2428.75},
"Surface_moy_maison": {"2020": 92.22, "2021": 94.39, "2022": 92.34, "2023": 93.39, "2024": 94.72},
"Terrain_moy_maison": {"2020": 327.83, "2021": 338.75, "2022": 318.83, "2023": 330.83, "2024": 332.15},

"Prix_moy_appartement": {"2020": 114866.45, "2021": 131237.19, "2022": 139630.64, "2023": 137395.58},
"Prix_med_appartement": {"2020": 100825.0, "2021": 113650.0, "2022": 124000.0, "2023": 123000.0},
"Prix_m2_moy_appartement": {"2020": 2023.34, "2021": 2286.03, "2022": 2497.42, "2023": 2486.55},
"Prix_m2_med_appartement": {"2020": 1713.21, "2021": 1885.5, "2022": 2134.92, "2023": 2129.63},
"Surface_moy_appartement": {"2020": 61.5, "2021": 61.99, "2022": 59.6, "2023": 59.82, "2024": 59.7},

"evolution_prix_m2": {"debut_annee": 2020, "fin_annee": 2024, "pct": 25.67},
"tendance": "Très forte hausse",
"variation_annuelle_prix_m2": {"2020": null, "2021": 11.4, "2022": 13.14, "2023": -0.66, "2024": 0.38},
"annee_plus_forte_hausse_prix": "2022",
"annee_plus_forte_baisse_prix": "2023",
"periode_donnees": {"debut": 2020, "fin": 2024},
"evolution_ventes": {"debut_annee": 2020, "fin_annee": 2024, "pct": 240.54},
"variation_annuelle_ventes": {"2020": null, "2021": 441.29, "2022": -5.88, "2023": -21.54, "2024": -14.8}}

"population": {"2020": 139926, "2021": 139926, "2022": 139926, "2023": 139456, "2024": 139456},
"coordonnees": {"latitude_mairie": 48.4, "longitude_mairie": -4.5},
"Segmen_Commune": "Très grande ville",
"Prix_med_groupe_pop": {"2020": 2757.68, "2021": 2900.0, "2022": 3184.11, "2023": 3114.75, "2024": 918.57},
"Ecart_pct_prix_m2_vs_groupe": {"2020": -37.15, "2021": -32.44, "2022": -30.12, "2023": -29.41, "2024": -24.13},
"Quartile_prix_m2_dans_segment": 1,
"evolution_population": {"debut_annee": 2020, "fin_annee": 2024, "pct": -0.34},
"variation_annuelle_population": {"2020": null, "2021": 0.0, "2022": 0.0, "2023": -0.34, "2024": 0.0}}

```

Une fois le JSON construit, on peut commencer le chatbot.

Le fonctionnement du système de question-réponse repose sur une série d'étapes permettant de transformer une question en langage naturel en une réponse précise issue des données immobilières. Dans un premier temps, la question posée par l'utilisateur est analysée afin d'identifier le nom de la ou des communes mentionnées. Cette étape s'appuie sur une normalisation des noms (passage en majuscules, suppression des accents, gestion des abréviations comme "Saint" ou "Sainte") afin de fiabiliser la recherche. Une attention particulière est portée aux ambiguïtés : certaines communes peuvent exister dans plusieurs départements, et certains mots courants (comme "Eu" ou "Moyen") sont volontairement ignorés pour éviter des correspondances erronées.

Une fois les communes identifiées, le système extrait également des informations complémentaires présentes dans la question, notamment l'année et éventuellement le département. Si aucune année n'est précisée, le système sélectionne automatiquement la plus récente disponible dans les données. Dans le cas où une commune est présente dans plusieurs départements et que l'utilisateur ne précise pas lequel, le système demande une clarification donc l'ajout du numéro du département dans la question afin d'éviter toute confusion.

Les données correspondantes sont ensuite récupérées à partir du fichier JSON structuré par ville. Pour chaque commune identifiée, toutes les variables sont extraites, en tenant compte de l'année demandée lorsque cela est possible.

Le chatbot comporte également une étape de classification de la question visant à identifier l'intention. Les requêtes sont ainsi réparties en plusieurs catégories, telles que les questions générales, les questions sur les prix (globaux, maisons ou appartements), l'évolution du marché, le nombre de ventes, la population ou encore les surfaces. Cette classification s'appuie principalement sur la détection de mots-clés présents dans la question.

En fonction de la catégorie identifiée, seules les informations pertinentes sont sélectionnées afin de construire un contexte adapté, ce qui permet d'améliorer la précision et la pertinence de la réponse fournie. Les données sont alors transformées en un contexte textuel, sous forme de phrases décrivant les informations disponibles pour la commune. Ce contexte est spécifiquement ajusté selon le type de question posée, comme illustré ci-dessous avec l'exemple d'une classification de type « prix ».

```
sentences.extend(  
    [  
        prefix + f"le prix moyen d'un bien est de {pmoy:.0f} euros.",  
        prefix + f"le prix médian d'un bien est de {pmed:.0f} euros.",  
        prefix + f"le prix minimum enregistré est de {pmin:.0f} euros.",  
        prefix + f"le prix maximum est de {pmax:.0f} euros.",  
        prefix + f"les prix vont de {pmin:.0f} à {pmax:.0f} euros.",  
        prefix + f"le prix moyen au m² est de {p2m:.0f} euros.",  
        prefix + f"le prix médian au m² est de {p2med:.0f} euros.",  
        prefix + f"le prix au m² minimum est de {p2min:.0f} euros le m².",  
        prefix + f"le prix au m² maximum est de {p2max:.0f} euros le m².",  
        prefix + f"La fourchette des prix au m² va de {p2min:.0f} à {p2max:.0f} euros le m².",  
        prefix + f"L'écart-type des prix est de {ecart_str}.",  
    ]  
)
```

Une fois le contexte généré, celui-ci est transmis à un modèle de type BERT spécialisé en question-réponse, en l'occurrence le modèle CATIE-AQ/QAmemberta. Ce modèle analyse simultanément la question et le contexte afin d'identifier le segment de texte le plus susceptible de contenir la réponse. Il repose sur un mécanisme de prédiction de positions de début et de fin dans le texte, permettant d'extraire automatiquement la réponse la plus probable, accompagnée d'un score de confiance.

Afin d'améliorer la lisibilité, la réponse extraite est ensuite replacée dans une phrase complète issue du contexte. Cela permet de fournir une réponse plus naturelle et compréhensible pour l'utilisateur. Toutefois, dans certains cas, le modèle BERT peut ne pas parvenir à extraire une réponse satisfaisante (par exemple si l'information est absente ou mal formulée). Un mécanisme de fallback est alors mis en place : il repose sur des expressions régulières et des règles spécifiques permettant d'extraire directement certaines informations (prix, population, nombre de ventes, etc.) à partir du contexte.

Enfin, le système retourne une réponse structurée comprenant plusieurs éléments : une réponse courte, éventuellement une phrase complète de contexte, un score de confiance lorsque disponible, ainsi que des informations sur la source de la réponse (modèle BERT ou fallback). Le temps de traitement et le contexte utilisé sont également conservés pour des besoins d'analyse ou d'évaluation.

votre question

Quel est le prix moyen au m² à Lyon ?

Envoyer

Réponse brute :

4613 euros

Réponse phrase :

À LYON, le prix moyen au m² est de 4613 euros.

Source : BERT • Score : 0.92 • Temps : 2.15s

Masquer le contexte

Contexte utilisé :

À LYON, le prix moyen d'un bien est de 267646 euros. À LYON, le prix médian d'un bien est de 240000 euros. À LYON, le prix minimum enregistré est de 30000 euros. À LYON, le prix maximum est de 800000 euros. À LYON, les prix vont de 30000 à 800000 euros. À LYON, le prix moyen au m² est de 4613 euros. À LYON, le prix médian au m² est de 4500 euros. À LYON, le prix au m² minimum est de 310 euros le m². À LYON, le prix au m² maximum est de 18910 euros le m². À LYON, La fourchette des prix au m² va de 310 à 18910 euros le m². À LYON, L'écart-type des prix est de 133393 euros.

B. L'analyse d'annonce

Dans le secteur immobilier, la manière dont un bien est décrit joue un rôle essentiel dans son attractivité. Un même élément peut être perçu très différemment selon la formulation utilisée : par exemple, il est préférable de mentionner une « *fenêtre orientée rue* » plutôt que d'utiliser directement le terme « *bruyant* », qui peut rebuter un potentiel acheteur ou locataire. Dans cette optique, une fonctionnalité d'analyse de sentiment a été intégrée au projet. Elle permet d'évaluer automatiquement le ton global d'une annonce immobilière afin d'aider les professionnels à optimiser leurs descriptions.

Concrètement, cette fonctionnalité repose sur un modèle de traitement du langage naturel spécialisé dans la classification de sentiments en français. Le modèle utilisé est

cardiffnlp/camembert-base-tweet-sentiment-fr, basé sur l'architecture CamemBERT, elle-même dérivée de BERT et optimisée pour la langue française. Initialement entraîné sur des données de type tweets, ce modèle est particulièrement efficace pour analyser des textes courts à moyens, comme les descriptions d'annonces immobilières.

L'implémentation s'appuie sur la bibliothèque Hugging Face Transformers, via un pipeline de type *text-classification*. Lorsqu'un texte est fourni, celui-ci est d'abord nettoyé puis éventuellement tronqué à une longueur maximale de 2000 caractères afin de garantir des performances optimales et éviter les surcharges mémoire. Le modèle est ensuite appliqué avec le paramètre **top_k=None**, ce qui permet de récupérer l'ensemble des scores de probabilité pour chaque classe (positif, négatif, neutre), et pas uniquement la classe dominante.

En sortie, le système fournit trois éléments : le label du sentiment dominant, un score de confiance associé (compris entre 0 et 1) et un détail des probabilités pour chaque catégorie. Cette approche permet non seulement d'identifier rapidement la tonalité globale d'une annonce, mais aussi de nuancer l'analyse en observant les scores intermédiaires.



```
Response body
{
  "label": "neutral",
  "score": 0.9489956498146057,
  "details": [
    {
      "label": "neutral",
      "score": 0.949
    },
    {
      "label": "positive",
      "score": 0.0395
    },
    {
      "label": "negative",
      "score": 0.0116
    }
  ]
}
```

D'un point de vue technique, le chargement du modèle est optimisé en étant effectué au démarrage de l'API plutôt qu'à chaque requête. Le pipeline est ainsi initialisé une seule fois à l'ouverture du service, puis réutilisé pour toutes les requêtes suivantes. Cela permet de réduire les temps de réponse et d'améliorer les performances globales du système, en particulier dans un contexte d'utilisation interactive.

Le système a été conçu pour être robuste et facilement intégrable. Il gère automatiquement les textes trop longs, évite les problèmes liés aux entrées vides. Ainsi, cette fonctionnalité constitue une aide à la rédaction précieuse : elle ne remplace pas l'expertise humaine, mais agit comme un outil d'assistance permettant d'améliorer la qualité des annonces, d'en renforcer l'attractivité et, in fine, d'augmenter leur efficacité commerciale.

C. Le résumé de texte

Les professionnels sont également souvent confrontés à des textes très denses. Lire et extraire rapidement les informations clés peut prendre un temps précieux. La fonctionnalité de résumé automatique répond à ce besoin en générant des condensés clairs et exploitables à partir d'un texte complet.

Cette fonctionnalité repose sur un modèle de type seq2seq : `plguillou/t5-base-fr-sum-cnndm`, un T5 français fine-tuné pour la génération de résumés. Le modèle permet de produire un texte condensé qui conserve les informations importantes tout en réduisant la longueur. Pour optimiser les performances, le modèle et son tokenizer sont chargés dès le démarrage de l'API, puis conservés en mémoire et réutilisés pour l'ensemble des requêtes. Cette approche évite les temps de chargement répétés et garantit une meilleure réactivité. Si un GPU est disponible, le modèle peut également y être déployé et utilisé en demi-précision (FP16) afin d'accélérer l'inférence.

Le système permet de contrôler la longueur du résumé grâce au paramètre `longueur`, qui peut être `"court"`, `"moyen"` ou `"long"`. Cette valeur détermine un ratio appliqué sur le nombre de tokens du texte d'origine pour générer un résumé proportionné, avec des limites minimales et maximales pour éviter des sorties trop brèves ou trop longues. Il est également possible de fixer directement des bornes `min_length` et `max_length` si besoin.

Avant la génération, le texte est nettoyé et tronqué si nécessaire pour ne pas dépasser les limites de tokenisation. La génération utilise `model.generate` avec un faisceau de recherche (*beam search*), ce qui permet d'obtenir un résumé cohérent et déterministe tout en restant rapide. Si la génération échoue ou que le résultat est vide, la fonction renvoie un extrait tronqué du texte source ou un message d'erreur lisible, évitant ainsi de bloquer l'API ou l'application.

Cette approche offre aux professionnels de l'immobilier un outil puissant pour gagner du temps, synthétiser des rapports ou extraire rapidement les points essentiels d'un marché local. Elle s'intègre facilement dans une API ou un workflow existant, permettant de produire des résumés fiables directement exploitables pour la prise de décision ou la communication avec les clients.

D. Le résumé de PDF

En plus des textes bruts, les professionnels sont aussi souvent confrontés à des documents volumineux : rapports de marché, études de prix, etc. Nous avons donc choisi d'ajouter une fonctionnalité de résumé automatique spécifique aux documents PDF.

Le processus se déroule en deux étapes principales. Tout d'abord, l'extraction du texte est réalisée via la fonction `extraire_texte_pdf`, qui ouvre le PDF et récupère le texte page par page. Cette fonction accepte différentes formes d'entrée : un chemin de fichier, un flux `BytesIO` ou directement des bytes en mémoire. La bibliothèque PyMuPDF (`fitz`) est utilisée pour lire chaque page et concaténer le texte. La chaîne finale est nettoyée pour supprimer les espaces superflus. Si le PDF contient surtout des images scannées, l'extraction peut renvoyer une chaîne vide ou quasi vide : ce cas est explicitement géré pour informer l'utilisateur que le document n'est pas exploitable textuellement. Pour aller plus loin, le pipeline pourrait intégrer un OCR associé à un VLM afin de générer un résumé non seulement des textes extraits, mais aussi des informations pertinentes présentes dans les graphiques, tableaux ou images du document.

Ensuite, le texte extrait est transmis à la fonction `resumer_texte`, qui applique le même pipeline T5 que pour les chaînes brutes. A nouveau, le résumé peut être produit

selon trois longueurs prédéfinies (**court**, **moyen**, **long**) ou avec des bornes personnalisées en tokens.

L'ensemble de la chaîne est conçu pour être fiable et intégrable dans un workflow ou une API. En cas d'erreur lors de l'ouverture ou de la lecture du PDF, la fonction renvoie un message d'erreur explicite plutôt que de planter. De même, si aucun texte exploitable n'est trouvé, elle signale clairement que le PDF est vide ou composé d'images, ce qui permet au professionnel de réagir en conséquence.

E. La recherche d'informations textuelles

L'application intègre un système de recherche d'information textuelle avancé basé sur le principe du RAG (retrieval-augmented generation). Le but est de proposer à l'utilisateur de chercher des informations plus littéraires comme des définitions. Nous avons donc choisi différentes sources qui nous ont paru fiable pour créer une base de données de pdfs pour répondre à ces questions.

Le module permet d'exploiter des PDF immobiliers provenant de sources variées (data.gouv [1], Observatoire NAFU[3] ou publications de notaires[4]), en automatisant leur téléchargement, extraction du texte, et découpage en morceaux (chunks). Chaque chunk est ensuite représenté par un vecteur d'embedding, stocké dans un index FAISS, ce qui rend possible la recherche rapide des passages les plus pertinents en réponse à une question posée par l'utilisateur.

Lorsqu'une question est formulée, elle est normalisée et transformée en vecteur via le même modèle d'embedding que celui utilisé pour les chunks. L'index FAISS retourne alors les 5 documents les plus proches, que le système peut ensuite transmettre à un modèle seq2seq (type FLAN-T5) pour générer une réponse en français, synthétique et contextualisée, directement exploitable par l'utilisateur.

Grâce à ce mécanisme, les professionnels de l'immobilier peuvent interroger rapidement une grande quantité de documents et obtenir des réponses précises, tout en conservant la traçabilité des sources.

Si notre modèle est fonctionnel, les résultats qu'il fournit ne sont pas totalement à la hauteur de nos espérances. L'indexation des chunks provoque des ruptures de contexte du texte, ce qui engendre une perte d'information. Ainsi, le modèle rencontre particulièrement des difficultés à répondre correctement sur des questions concernant des dates. C'est donc une autre piste d'amélioration possible.

II. La mise en production

A. L'API

Le module 06_FastAPI.py fournit une API REST permettant d'interagir avec l'Assistant Immobilier pour obtenir des réponses à des questions, résumer des documents ou analyser des textes. Lors de son initialisation, il charge le chatbot à partir d'un fichier JSON contenant les données immobilières DVF et, si disponible, le moteur RAG PDF pour effectuer des recherches sémantiques dans des documents PDF indexés.

Concrètement, lorsqu'une question est posée via l'endpoint `/ask`, l'API transmet la requête au chatbot, qui recherche la réponse dans le JSON et renvoie le texte correspondant, éventuellement accompagné de la source et d'un score de pertinence. Pour les PDF, l'endpoint `/ask_pdf` interroge le moteur RAG, qui compare la question avec les vecteurs de texte extraits des documents et retourne les extraits les plus proches sémantiquement. Les endpoints `/summarize` et `/summarize_pdf` utilisent les fonctions de résumé pour condenser un texte ou un PDF selon une longueur définie, tandis que `/sentiment` analyse un texte (par exemple une annonce immobilière) pour déterminer son ton positif, négatif ou neutre, avec un score de confiance et la répartition par classe.

L'API gère également l'état de ses ressources via l'endpoint `/health`, indiquant si le chatbot et le moteur RAG sont prêts, et signale toute indisponibilité. L'ensemble du système permet donc de combiner interrogation structurée des données, recherche sémantique dans des documents volumineux, synthèse de contenus et analyse de sentiment de manière automatisée et accessible via des requêtes HTTP standard.

B. L'application Streamlit

L'interface est un point à ne pas négliger dans ce type de projet puisqu'il sert de point d'accès utilisateur à l'Assistant Immobilier SAFIR (Système d'Analyse des Flux Immobiliers et Résidentiels). Il faut que l'interface soit claire et intuitive pour qu'un nouvel utilisateur puisse directement trouver ce qu'il recherche.

L'interface ne fait aucun calcul côté navigateur : toutes les requêtes sont envoyées à l'API FastAPI via des POST JSON, avec timeout et désactivation des proxies pour des connexions fiables.

Niveau design, l'application est organisée en plusieurs écrans accessibles depuis une sidebar : accueil, questions numériques, analyse de sentiment d'annonces, recherche sémantique sur PDF, et résumés de texte ou de PDF. La navigation est gérée par `st.session_state["page"]`, permettant de conserver l'état entre les écrans.

Côté visuel, l'UI utilise une mise en page centrée, une icône maison et un thème bleu saphir pour les boutons. Les résultats ou messages d'erreur réseau sont affichés dynamiquement via `st.error`.

En résumé, cette interface multi-pages offre une navigation simple et intuitive, délègue toute la logique métier à l'API et permet aux professionnels de l'immobilier d'accéder facilement aux fonctionnalités et modèles du projet.

C. Le docker deployment ready

Pour ce projet, Docker est utilisé afin de packager l'API et l'interface Streamlit dans un environnement stable et reproductible. L'avantage principal est de pouvoir lancer le projet sans avoir à installer manuellement toutes les dépendances lourdes comme PyTorch, FAISS ou PyMuPDF, tout en garantissant la même configuration pour tous les utilisateurs.

L'image Docker de l'API est basée sur **python:3.11-slim**. Elle installe les paquets système nécessaires, copie le code Python et les données DVF, expose le port 8001 et centralise les modèles Hugging Face dans le répertoire **/app/.cache/huggingface**. L'API est ensuite démarrée avec **Uvicorn**, et les fichiers optionnels comme l'index FAISS peuvent être montés depuis l'hôte pour permettre la recherche sémantique sur les PDF, sans être inclus directement dans l'image.

Grâce à Docker Compose, il est possible de lancer simultanément l'API et l'interface Streamlit. L'API écoute sur le port 8001, avec le cache Hugging Face persistant entre les conteneurs, tandis que Streamlit s'exécute sur le port 8501 et communique avec l'API via le réseau interne Docker. Compose garantit également que l'API démarre avant l'interface pour un fonctionnement correct.

En résumé, Docker assure un déploiement simple, reproductible et flexible, tout en centralisant le cache des modèles et en connectant facilement l'API à l'interface Streamlit, ce qui facilite grandement l'utilisation et le partage du projet.

Malgré les avantages offerts par Docker, cette étape a présenté quelques difficultés. L'installation de Docker Desktop s'est révélée relativement complexe, la connexion avec le noyau a parfois posé problème, et nous avons rencontré des erreurs inexpliquées, notamment une indisponibilité temporaire de l'API. Nous avons finalement fini par résoudre ce problème mais la gestion de celui-ci nous a coûté beaucoup de temps.

Conclusion

Le projet SAFIR illustre comment il est possible de combiner données immobilières, calculs numériques et traitements NLP pour créer un assistant intelligent destiné aux professionnels du secteur. L'outil centralise plusieurs fonctionnalités : consultation des prix moyens et tendances du marché, analyse de sentiment des annonces, génération de résumés de textes et de PDF, et recherche sémantique via FAISS, le tout orchestré par une API FastAPI performante. L'interface Streamlit, intuitive et responsive, et l'environnement Docker reproductible permettent une utilisation fiable et flexible sur différentes machines, garantissant un accès simple et rapide à des informations complexes.

Ce projet nous a permis de renforcer notre compréhension des pipelines NLP, de la gestion de données volumineuses et hétérogènes, ainsi que de la mise en place d'API robustes et d'interfaces utilisateur interactives. Il a aussi mis en évidence des axes d'amélioration : intégrer des traitements OCR et multimodaux (VLM) pour exploiter le contenu visuel des PDF, enrichir la recherche sémantique avec des modèles plus puissants ou spécialisés, et optimiser la performance globale pour gérer des volumes de données plus importants. SAFIR constitue ainsi une base solide pour développer des assistants intelligents toujours plus complets et précis dans le domaine immobilier.

Bibliographie

[1] Foire aux question et notice descriptive des fichiers DVF, data.gouv.fr

Lien original : <https://www.data.gouv.fr/datasets/demandes-de-valeurs-foncieres>

Liens d'API pour les fichiers concernés :

<https://www.data.gouv.fr/api/1/datasets/r/d573456c-76eb-4276-b91c-e6b9c89d6656>

<https://www.data.gouv.fr/api/1/datasets/r/087ec735-74fd-48a7-a82e-0b1cd3ea6fe9>

[2] <https://www.data.gouv.fr/datasets/demandes-de-valeurs-foncieres>

[3] Marché immobilier (DVF), Observatoire NAFU

https://observatoire-nafu.fr/espaces_nafu/marche-immobilier/

[4] Les notes de conjoncture immobilière, [Notaires.fr](https://www.notaires.fr)

<https://www.notaires.fr/fr/immobilier-fiscalite/prix-et-tendances-de-limmobilier/les-notes-de-conjoncture-immobiliere>